AI DIGITAL OFFICE OS v1.3 ××× Consolidated Master

Gap-Closed, Research-Backed Master Enterprise Blueprint, Full Data Model, Full API Contract, Sequence Diagrams, NFR/SLO Targets & Measurable Acceptance Checklist

Consolidated Complete Package — Standalone system, no relation to AL FATAH SMART POS

Finalized 2026-08-29

Package Manifest — v1.3

#	File	Section
1	README.md	A
2	AI_DIGITAL_OFFICE_OS_MASTER_BLUEPRINT_v1.3.md (58 clauses)	B
3	database_schema_v1.3.sql (27 tables)	C
4	control_plane_openapi_v1.3.yaml (18 endpoints)	D
5	architecture_diagrams_v1.3.md (3 original + 4 new sequence diagrams)	E
6	agent_specification_template_v1.3.md	F
7	nfr_slo_v1.3.md (new)	G
8	implementation_acceptance_checklist_v1.3.md (measurable)	H
9	CHANGELOG_v1.2_to_v1.3.md (honest diff)	I

Section A — README (v1.3)

AI DIGITAL OFFICE OS v1.3 — COMPLETE PACKAGE (Gap-Closed, Research-Backed Edition)

This package is the v1.3 update to the v1.2 baseline. It is produced specifically to close every structural gap identified in the v1.2 technical review: narrative vs. schema/API mismatch, missing multi-tenancy, missing RBAC tables, missing artifact/message/memory/workflow tables, a skeleton OpenAPI contract with no security scheme, no concrete NFRs/SLOs, and no sequence diagrams.

This is a standalone update to the AI DIGITAL OFFICE OS specification only. It has no relationship to, and does not include, the separate AL FATAH SMART POS project — the blueprint explicitly reaffirms these are two different software systems (see blueprint clause 34).

Main deliverables

1. `AI_DIGITAL_OFFICE_OS_MASTER_BLUEPRINT_v1.3.md` — full spec, clauses 1–58 (1–40 preserved from v1.2 verbatim, 41–58 new)
2. `database_schema_v1.3.sql` — full PostgreSQL logical schema (27 tables, multi-tenant, RBAC, workflow durability, agent messaging, tiered memory/pgvector, artifact lineage, secrets references, feature flags, indexes, baseline RLS policies)
3. `control_plane_openapi_v1.3.yaml` — full OpenAPI 3.0.3 contract (18 endpoints, JWT bearer auth, 15 component schemas, standard error envelope, /v1 versioning)
4. `architecture_diagrams_v1.3.md` — 3 original diagrams preserved + 4 new sequence/structure diagrams (task routing, agent activation, approval escalation, multi-tenant isolation)
5. `agent_specification_template_v1.3.md` — updated with tenant scoping, MCP tool declarations, idempotency requirement
6. `nfr_slo_v1.3.md` — **new file**: concrete latency/availability/retry/RTO-RPO/cost numbers (previously policy language only)
7. `implementation_acceptance_checklist_v1.3.md` — every item now has a measurable pass condition, not a bare checkbox
8. `CHANGELOG_v1.2_to_v1.3.md` — honest, itemized diff of what changed and why

Baseline

The v1.2 package (including its two source PDFs) remains the architectural baseline and is not removed or contradicted — v1.3 only adds the missing depth clauses 41–58 needed to make the v1.2 narrative and its schema/API actually consistent with each other.

Build order

Blueprint (clauses 1–58) → database_schema_v1.3.sql → control_plane_openapi_v1.3.yaml → architecture_diagrams_v1.3.md → agent_specification_template_v1.3.md → nfr_slo_v1.3.md → implementation_acceptance_checklist_v1.3.md. This is the recommended reading/build order for a coding agent implementing the system.

Important — read this before treating v1.3 as “final”

This remains a technical blueprint and implementation baseline, not running code. It intentionally does NOT include (and no version should fabricate): actual infrastructure-as-code, cloud account/network topology, real provider contracts or pricing agreements, penetration-test results, or production credentials. These are environment-specific choices that must be made during implementation. See blueprint clause 58 (“Explicit Non-Goals”) for the full, honest list.

Section B — Master Enterprise Blueprint v1.3 (Full Text, Clauses 1–58)

AI DIGITAL OFFICE OS

Autonomous Agentic Software Factory & Organization Operating System

MASTER ENTERPRISE BLUEPRINT v1.3

ENTERPRISE TECHNICAL & IMPLEMENTATION SPECIFICATION — GAP-CLOSED, RESEARCH-BACKED EDITION

0. Document Control

- Baseline: AI DIGITAL OFFICE OS Master Enterprise Blueprint v1.2 (itself baselined on v1.1).
- Target: Version 1.3 — closes every structural gap identified in the v1.2 review (narrative/schema mismatch, missing multi-tenancy, missing RBAC tables, missing artifact/message/memory/workflow tables, thin OpenAPI contract, no concrete NFRs/SLOs, no sequence diagrams).
- Status: Production-grade architectural, data-model, API and operational baseline. This version is intended to be **locked** and used as the source for an implementation master prompt.
- Scope: Every clause from v1.2 (1–40) is preserved verbatim below with no content removed, no business logic weakened, and no re-design of the approved hierarchy. New clauses 41–58 add the missing depth; they do not contradict 1–40.
- Research basis: Clauses 41–58 incorporate patterns confirmed as current industry practice as of August 2026 — agent-to-agent interoperability protocols (A2A over HTTP/JSON-RPC, MCP for tool discovery), durable-execution workflow engines (Temporal-style activity/retry semantics), composite-key tenant-scoped RBAC (tenant_id carried into every junction table to prevent cross-tenant privilege escalation), and pgvector HNSW indexing for agent memory.
- Core principles (unchanged): Model-Agnostic, Provider-Agnostic, Agent-Extensible, Tool-Extensible, Secure, Auditable, Scalable, Recoverable, Testable. **Added for v1.3:** Tenant-Isolated, Interoperable (MCP/A2A), Measurably Reliable (concrete SLOs, not just policy language).

1. Executive System Definition

- AI DIGITAL OFFICE OS is a governed AI organization and autonomous software factory. It receives a business objective, decomposes it into governed work, assigns specialist agents, selects appropriate AI capabilities, invokes controlled tools, produces versioned artifacts, verifies outputs independently, and releases approved results.
- The system is not a single chatbot and not a collection of disconnected agents. It is an operating system for coordinated AI work with hierarchy, durable workflows, policies, memory, artifacts, security, quality gates, observability, and human authority.
- Authoritative business calculations and invariants remain in deterministic domain services. AI decides and orchestrates; deterministic engines calculate and enforce business rules.

2. Approved v1.1 Architecture to Preserve

- Layer 1: Owner & Human Governance.
- Layer 2: Executive Management: Master CEO, Executive Manager, Assistant Manager.
- Layer 3: Orchestration Control Plane: stateful workflows, dependency DAG, task dispatch, deadlock detection and run tracking.
- Layer 4: Agent Factory & Registry: specification, cloning/specialization, sandboxing, test generation, benchmarking, capability profiling and lifecycle.
- Layer 5: Specialized Agents: product, research, architecture, UI/UX, frontend, backend, QA, security, DevOps, prompt/creative and locked AL FATAH operational agents.
- Layer 6: AI Provider Gateway & Router: provider adapters, model registry, capability classes, intelligent routing and controlled credentials.
- Layer 7: Tool Gateway & Services: MCP, Git, shell, Docker, Postgres, browsers, web/search and automation services under RBAC.
- Layer 8: Memory, Artifacts & Audit: pgvector knowledge, project memory, immutable artifacts, QA, security scanning, telemetry and audit trails.
- **v1.3 addition — Layer 9: Identity & Tenancy:** tenants/organizations, users, tenant-scoped roles and permissions, session management — sits beneath Layer 1 as the isolation boundary every other layer must respect.

3. Executive Command Hierarchy

- Owner/Chairman is the final authority for strategy, RED operations, budget overrides, destructive production actions and emergency kill-switch control.
- Master CEO converts intent into goals, project constitutions, milestones and quality gates.
- Executive Manager performs work breakdown, departmental allocation and scheduling.
- Assistant Manager coordinates execution, dependencies, artifact verification and next-agent triggering.
- Departments execute typed tasks through governed agents. No department bypasses the orchestration and policy layers for critical work.

4. Agent Factory

- Agent Factory is a first-class subsystem responsible for creating, cloning, extending, specializing, evaluating, activating, upgrading, deprecating and retiring agents.
- Creation pipeline: Request → Requirement Analysis → Capability Design → Agent Specification → Prompt/Instruction Generation → Tool Selection → Permission Generation → Model Policy → Input/Output Schemas → Test Generation → Sandbox → Evaluation → Security → Approval → Registry → Activation.
- Generated agents must start in a sandbox and must not receive unrestricted production credentials, root access or unrestricted database access.
- Agent cloning and specialization preserve lineage through parent_agent_id and independent versions.
- **v1.3 addition:** the Agent Factory is itself a governed agent — it cannot activate an agent into production; it can only advance an agent to `APPROVED`. Activation is a separate, human- or policy-triggered transition (see clause 45).

5. Agent Lifecycle & Safety

- Lifecycle states: DRAFT, SANDBOX, TESTED, EVALUATED, APPROVED, ACTIVE, UPDATED, DEPRECATED, RETIRED.
- Promotion requires the applicable quality and security gates. An agent may be automatically created but production activation remains governed.
- Agent creation must be bounded. No agent may recursively create unlimited agents without Agent Factory, Policy Engine and approval controls.
- **v1.3 addition:** hard ceiling of 25 active agents per department and 200 active agents per tenant by default (configurable per-tenant policy, itself an approval-gated change) to prevent uncontrolled agent sprawl.

6. Agent Registry Technical Contract

- Required identity: agent_id, name, department, role, purpose, version, lifecycle_state.
- Capability contract: capabilities, required_model_capabilities, preferred_provider, preferred_model, fallbacks.
- Execution contract: allowed_tools, permissions, data_access, sandbox_policy, input_schema, output_schema.

- Operational contract: evaluation_score, success_rate, average_latency, average_cost, status, created_at, updated_at, parent_agent_id.
- Registry is authoritative for what an agent is allowed to do. It is not merely a directory.
- **v1.3 addition:** every agent record is `tenant_id`-scoped (see clause 41); a `NULL tenant_id` marks a system/global agent template that a tenant may clone into its own scope but never execute directly.

7. AI Provider Gateway

- All external AI access passes through a central AI Provider Gateway. Agents do not directly hold unrestricted provider API keys.
- Provider adapters isolate vendor-specific APIs from the core operating system. Initial adapters may include OpenAI/GPT, Anthropic/Claude, Google/Gemini, Moonshot/Kimi, OpenAI-compatible APIs and local model runtimes.
- Adding a future provider follows: Adapter → Registration → Capability Mapping → Health Probe → Security Review → Evaluation → Approval → Production.
- Provider outages, rate limits and transient errors must not halt the Office when an approved compatible fallback exists.

8. Provider Registry

- Provider entity fields: provider_id, provider_name, provider_type, adapter_type, protocol, endpoint, authentication_method, supported_capabilities, supported_models, rate_limits, quota_rules, pricing_rules, privacy_classification, data_residency, availability, health_status, version and timestamps.
- Provider identity is separate from model identity so models can change independently of provider integration.

9. Model Registry & Capability System

- Model registry stores model identity, provider relationship, version, context window, supported input/output types, tool calling, structured output, vision, coding, reasoning, research, latency, cost, privacy and health.
- Agents request capabilities rather than hard-coding model names. Example capability classes: HEAVY_CODING, DEEP_RESEARCH, REASONING, VISION_UI, LONG_CONTEXT, FAST_ROUTINE, LOW_COST, HIGH_RELIABILITY, STRUCTURED_OUTPUT, TOOL_USE, LOCAL_OFFLINE.
- Model Registry, pricing, health and evaluation records are independently maintainable.

10. Intelligent Model Router

- Routing inputs: task, agent, required capability, project policy, security classification, data classification, provider health, model health, quality, cost, latency, context requirements, tool requirements and owner override.
- Every routing decision is auditable: task_id, agent_id, requested capability, selected provider, selected model, reason, policy result and timestamp.
- Fallback chain: Primary → Approved Secondary → Approved Local Engine → Human Escalation. Fallback must preserve capability, privacy and security requirements.

11. Model Evaluation Engine

- Every production model should have standardized benchmarks covering coding, reasoning, research, tool use, structured output, vision, reliability, latency, cost efficiency, instruction following, security and privacy compliance.
- Evaluation results are versioned and may be used by the router. Models can be re-evaluated as providers change versions or pricing.

12. Central Policy Engine

- Policy Engine is a first-class service. It evaluates Agent + Task + Data + Tool + Model + Provider + Environment + Risk and returns ALLOW, DENY, REQUIRE_APPROVAL or REQUIRE_ESCALATION.
- Policies govern permissions, tool access, data access, model/provider selection, production operations, external API calls, sensitive information, deployment and destructive operations.
- Policies are machine-readable, versioned, auditable and applied before execution.

13. RBAC & Least Privilege

- Authorization chain: Agent → Role → Permission → Tool → Resource → Action.
- Support role, resource, action, environment and project scoped permissions. Typical actions include READ, WRITE, CREATE, UPDATE, DELETE, EXECUTE, DEPLOY, APPROVE, EXPORT and ADMINISTER.
- Default posture is deny. Permissions are explicitly granted and scoped.
- **v1.3 — full data-model closure, see clause 44.**

14. Approval Engine & Risk Tiers

- Approval flow: Request → Risk Classification → Policy Check → Approval Routing → Decision → Execution → Audit.
- GREEN permits autonomous actions within policy. YELLOW requires designated management review. RED requires Owner or explicitly authorized human sign-off.
- Approval records include request_id, requester, agent, action, risk, reason, approver, decision, timestamp, expiration and execution result.

15. Data Governance

- Data classifications: PUBLIC, INTERNAL, CONFIDENTIAL, SENSITIVE, RESTRICTED.
- Each class defines allowed agents, tools, models, providers, cloud/local rules, logging restrictions, export restrictions, retention and encryption requirements.
- Sensitive or restricted information must not be sent to an external model unless explicitly allowed by policy.

16. Secrets Management

- Use a dedicated Secrets Management Service for AI keys, database credentials, infrastructure credentials, OAuth secrets, signing keys and deployment credentials.
- Agents never receive unrestricted raw secrets. Use scoped and preferably short-lived credentials. Every secret access is auditable.
- **v1.3 — full data-model closure, see clause 46.**

17. Task Engine

- Every task has task_id, project_id, workflow_id, parent_task_id, assigned_agent, required_capability, priority, risk, security_level, dependencies, input, expected_output, status, retry_count, deadline and timestamps.
- Task states: CREATED, QUEUED, ASSIGNED, RUNNING, WAITING, BLOCKED, FAILED, RETRYING, COMPLETED, CANCELLED, ESCALATED.

18. Durable Workflow Engine

- Every critical workflow supports start, pause, resume, retry, cancel, escalate, rollback and completion.
- Workflow state must survive process restarts. Critical workflows must not depend solely on volatile in-memory state.
- **v1.3 — full data-model closure, see clause 43.** Workflow execution follows durable-execution semantics (event-sourced history, replayable state, per-activity retry policy) consistent with current production practice for long-running agent workflows.

19. Agent-to-Agent Communication

- Critical agent communication uses typed messages rather than uncontrolled conversational state.
- Message contract: task_id, workflow_id, sender_agent, receiver_agent, message_type, purpose, priority, security_level, input_schema, expected_output, dependencies, deadline, artifact_reference, status, result and error.
- **v1.3 — full data-model closure, see clause 42.** Cross-boundary agent communication (to external/partner agent systems) additionally follows an A2A-style contract: HTTP+JSON-RPC transport, agent capability cards, and task lifecycle negotiation — kept optional and disabled by default until an explicit integration is approved.

20. Dependency DAG

- Typical chain: Research → Product → Requirements → Architecture → UI/UX → Development → Review → QA → Security → Deployment → Monitoring.
- DAG engine detects circular dependencies, deadlocks, missing dependencies, failed dependencies, blocked tasks and timeout conditions.

21. Prompt Registry

- Prompt Registry stores prompt_id, agent_id, version, system instructions, variables, input/output contracts, compatible capabilities, evaluation score, security classification, author, timestamps, changelog and rollback version.
- Prompt changes are versioned and must pass regression evaluation before production use.

22. Knowledge & Memory

- Memory classes: working memory, task memory, agent memory, project memory, organizational memory, decision memory and artifact memory.
- Retrieval flow: Query → Authorization → Retrieval → Filtering → Ranking → Context Assembly → Agent → Source/Citation Reference.
- Memory ownership, retention, access, versioning and deletion are policy controlled.
- **v1.3 — full data-model closure, see clause 47.** Retrieval is tiered: hot/working memory in a fast key-value layer,

durable structured memory in relational tables, and semantic memory in pgvector with HNSW indexing — routed by access pattern rather than sending every lookup through vector search.

23. Artifact Lineage

- Full lineage: Owner Command → Project → Task → Agent Run → Model Run → Tool Calls → Artifact → Review → Approval → Git Commit → Build → Deployment → Release.
- Important artifacts must be immutable/versioned and traceable to their originating workflow and decision context.
- **v1.3 — full data-model closure, see clause 48.**

24. Decision Records

- Maintain Architecture/Decision Records for important choices. Store decision_id, project_id, decision, reason, alternatives, evidence, agent, model, author, approval, timestamp, impact and status.

25. Git & Engineering Control

- Git is a controlled subsystem supporting branches, commits, pull requests, code review, tags, releases and rollback.
- Production branches are protected. Agents operate with scoped repository permissions. Production deployment cannot originate from unreviewed changes.

26. Software Delivery Pipeline

- Development → Unit Test → Integration Test → Static Analysis → Security Scan → Code Review → QA → Staging → Approval → Production → Health Check → Monitoring.
- Production failures support policy-controlled automated rollback.

27. Deployment Strategies

- Support standard, rolling, blue/green and canary deployment strategies.
- Every deployment has version, health checks, readiness checks, approval state, audit trail and rollback trigger.

28. Disaster Recovery

- Define database, artifact, configuration and backup policies, point-in-time recovery, restore testing, RPO and RTO.
- Recovery procedures must be tested periodically and their results recorded.
- **v1.3 — concrete targets, see clause 50.**

29. Observability

- Unified logs, metrics, traces, events and alerts cover agents, tasks, workflows, models, providers, tools, APIs, databases, deployments, costs and errors.
- Every important request carries a correlation ID for end-to-end tracing.
- **v1.3 addition:** agentic workflows require observability beyond standard APM — non-deterministic agent decision paths (which model was chosen, why, what the agent considered and rejected) must be captured as structured trace events, not just latency/error metrics.

30. Cost Governance

- Track cost per request, task, agent, project and provider, plus daily and monthly budgets.
- Support warning, soft limit, hard limit and Owner approval. Automatic low-cost routing is allowed only when policy and quality requirements remain satisfied.

31. Error Handling & Reliability

- Standard error classes: VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, TOOL_ERROR, MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT, DATABASE_ERROR, ARTIFACT_ERROR, DEPLOYMENT_ERROR, UNKNOWN_ERROR.
- Each error records error_id, task/workflow/agent, service, severity, message, retryability, retry count, recovery action and timestamp.
- Critical operations are idempotent. Retries use bounded exponential backoff and dead-letter handling where appropriate.
- **v1.3 — concrete retry/backoff numbers, see clause 49.**

32. Controlled Self-Healing

- Existing bounded self-healing remains: Failure → Root Cause Analysis → Patch Agent → Regression Verification.
- Maximum automatic attempts remain strictly bounded at three before human escalation. Self-healing must not bypass governance or create uncontrolled duplicate production actions.

33. Project Factory

- Universal lifecycle: IDEA → DISCOVERY → RESEARCH → REQUIREMENTS → PRODUCT PLAN → ARCHITECTURE → UX/UI → DESIGN → DEVELOPMENT → CODE REVIEW → QA → SECURITY → STAGING → APPROVAL → DEPLOYMENT → MONITORING → MAINTENANCE.
- Supported product types include websites, web applications, mobile applications, APIs, SaaS, dashboards, POS, CRM, ERP, internal tools, automation systems, AI applications and future software categories.

34. AL FATAH SMART POS

- AL FATAH SMART POS remains the first major factory implementation. Its locked operational agents remain domain workers under the Digital Office.
- The same Agent Factory, Model Router, Tool Gateway, QA, Security, Artifact and Governance systems are reusable for POS and future factories.
- **Explicit clarification carried over from user direction:** AI DIGITAL OFFICE OS and AL FATAH SMART POS are two separate software systems. This blueprint governs the Digital Office only; POS-specific business logic is out of scope here and is not to be merged into this schema or API.

35. Multi-Tenancy Readiness

- Even if the first deployment is single-organization, the data model and authorization boundaries should be ready for organizations, teams, users, roles and isolated projects without a complete core rewrite.
- **v1.3 — this is no longer aspirational. Multi-tenancy is implemented in the v1.3 schema from day one; see clause 41.**

36. Configuration & Feature Flags

- Operational configuration is versioned, environment-specific, auditable, validated and rollback-capable.
- Feature flags support staged activation of agents, models, providers, tools, workflows and experimental capabilities.
- **v1.3 — full data-model closure, see clause 51.**

37. Architecture Consistency Rules

- Every agent has governance, permissions and evaluation. Every permission maps to policy. Every model belongs to a provider. Every provider has an adapter. Every task belongs to a workflow/project. Every critical action has a risk level and approval path. Every artifact has lineage. Every production deployment has rollback. Every secret has controlled access. Every important event is observable.
- **v1.3 addition:** every tenant-scoped table carries `tenant_id`; every junction table carries `tenant_id` alongside its foreign keys so referential integrity alone cannot produce a cross-tenant privilege escalation (composite-key enforcement, not just application-layer checks).

38. Implementation Phases

- Phase 0: Constitution and governance.
- Phase 1: Core infrastructure and control plane.
- Phase 2: Agent Registry, Factory, sandbox and evaluation.
- Phase 3: MCP and Tool Gateway.
- Phase 4: Provider Gateway, Provider Registry, Model Registry and Router.
- Phase 5: Core engineering departments.
- Phase 6: Knowledge, memory, artifacts and telemetry.
- Phase 7: Advanced autonomy, cloning, specialization and bounded self-healing.
- Phase 8: Full Project Factory and AL FATAH SMART POS production launch.
- Phase 9+: Future scale, multi-tenancy, advanced deployment and provider/model expansion.
- **v1.3 note:** because multi-tenancy, RBAC, memory, artifacts, messaging and workflow durability are now in the Phase 1–2 schema instead of deferred to Phase 9+, the phase plan is unchanged in name but Phase 1 is now heavier — this is intentional and prevents the rewrite risk flagged in the v1.2 review.

39. Acceptance Test Matrix

- Architecture: all v1.1 components preserved and connected.
- Agents: creation, sandbox, evaluation, approval, activation, upgrade and retirement work.
- Providers: adapter registration, health check, capability mapping, security and evaluation work.

- Models: registry, benchmarks, pricing, health and router selection are auditable.
- Security: least privilege, secrets isolation and environment separation pass.
- Workflows: durable state, dependencies, retry, escalation and rollback pass.
- Delivery: code review, QA, security scan, staging and production gates pass.
- Operations: observability, cost controls, backups and disaster recovery are demonstrable.
- Extensibility: adding a provider, model, agent or product type does not require core architecture redesign.
- **v1.3 — see clause 53 for the fully measurable version of this matrix (pass/fail criteria with numbers, not just checkboxes).**

40. Final Enterprise Principle

- AI DIGITAL OFFICE OS is not tied to a single model, provider, agent, tool or software project. It is a reusable AI operating system and software factory capable of dynamically selecting AI capabilities, safely creating specialist agents, integrating future providers, orchestrating complete software projects and maintaining human-controlled governance throughout the lifecycle.

NEW IN v1.3 — Clauses 41 through 58 (Gap Closure)

41. Identity & Multi-Tenancy Model

- Tenancy pattern chosen: **shared database, shared schema, `tenant_id` column on every tenant-scoped table** (the standard scalable pattern for tens of thousands of tenants, versus one-database-per-tenant or one-schema-per-tenant which do not scale past low hundreds).
- Every tenant-scoped table's primary/foreign keys are paired with `tenant_id` in composite foreign keys or enforced via row-level security (RLS) policies, so referential integrity alone cannot create a cross-tenant reference — this closes the single most common multi-tenant RBAC bug (a syntactically valid but cross-tenant foreign key pairing).
- `organizations` (tenant root) → `users` (membership scoped to tenant, a user may belong to more than one tenant via a membership row) → `roles` (system template roles plus tenant-defined roles) → `permissions` (system-wide, tenant-agnostic definitions of "what the software can do") → `role_permissions` and `user_roles` (both tenant-scoped junctions).
- `NULL tenant_id` is reserved exclusively for system-level template rows (agent templates, prompt templates, default roles) that tenants clone into their own scope; no operational data may carry a NULL tenant_id.

42. Agent-to-Agent Message Bus (Data Model Closure)

- `agent_messages` table implements clause 19's message contract as durable, queryable rows rather than in-memory conversational state: task_id, workflow_id, sender_agent_id, receiver_agent_id, message_type, purpose, priority, security_level, input_payload, expected_output_schema, dependencies, deadline, artifact_reference, status, result, error, tenant_id, created_at, delivered_at, acknowledged_at.
- Message delivery is at-least-once with idempotency keys; receivers must be idempotent on `message_id`.
- Optional external interoperability: an `a2a_capability_cards` table publishes this Office's agents as A2A-discoverable capability cards for approved partner-system integration, disabled by default (policy-gated).

43. Durable Workflow Engine (Data Model Closure)

- `workflow_registry` stores workflow_id, project_id, tenant_id, workflow_type, definition_version, current_state, status (RUNNING/PAUSED/COMPLETED/FAILED/CANCELLED/ESCALATED), started_at, updated_at, completed_at.
- `workflow_history` stores an append-only, replayable event log per workflow (event_id, workflow_id, sequence_no, event_type, payload, created_at) — this is what makes workflow state survive a process restart: on recovery, the engine replays `workflow_history` to reconstruct `current_state` rather than trusting only the mutable row.
- Each workflow step maps to one or more `task_registry` rows; each task execution attempt is a retryable "activity" with its own bounded retry policy (see clause 49), consistent with current durable-execution engine practice (e.g., Temporal-style activity/retry semantics) — the Office does not need to adopt a specific third-party engine, but the schema is compatible with one if adopted later.

44. RBAC — Full Data Model Closure

- Tables: `organizations`, `users`, `user_organization_membership`, `roles`, `permissions`, `role_permissions`, `user_roles` (all tenant-scoped except the system-wide `permissions` catalog).
- Policy decisions fixed explicitly (per current best practice, these must be decided up front, not left to ad-hoc convention): role ownership = system template roles + tenant-defined extensions; permission catalog is system-wide and version-controlled; a role is unusable until at least one permission is attached; every `user_roles` row carries `tenant_id` even though it is derivable, specifically so the foreign keys are composite and a cross-tenant row is rejected by the database itself, not just by application code.
- Enforcement happens at two layers: coarse authentication + tenant-membership check at the API gateway, fine-grained

permission check at the service layer before any tool/model/data access — matching clause 13's chain (Agent → Role → Permission → Tool → Resource → Action), extended to also cover human users of the control plane, not only agents.

45. Agent Activation Authority (Closes the Factory/Activation Ambiguity)

- Agent Factory pipeline output stops at `APPROVED`. A separate, explicit `activation_requests` record (subset of `approval_requests`, action type `AGENT_ACTIVATE`) is required to transition `APPROVED → ACTIVE`. This guarantees no pipeline — however automated — can put an agent into production without passing through the Approval Engine defined in clause 14.

46. Secrets Management — Full Data Model Closure

- The Office **never stores raw secret values** in its own database. `secrets_vault_references` stores only: reference_id, tenant_id, secret_name, vault_path (pointer into an external secrets manager such as a KMS/Vault product), scope (which agent_id/provider_id/tool it is bound to), rotation_policy, last_rotated_at, expires_at, created_by, created_at.
- Every dereference of a `vault_path` by an agent is written to `audit_events` with actor, purpose, and resource — satisfying clause 16's "every secret access is auditable" without ever placing a raw credential in an application table.
- Default credential lifetime: 15 minutes for AI provider keys issued to a running agent task, 24 hours for tool-gateway service credentials, both revocable immediately on policy violation.

47. Knowledge & Memory — Full Data Model Closure

- Three tiers, matching current agent-memory architecture practice:
  1. **Working memory** (`working_memory_cache`) — short-lived, per-task-run scratch state, TTL-bound (default 4 hours), never used for durable facts.
  2. **Durable structured memory** (`memory_facts`) — long-lived facts, preferences, project/organizational memory: memory_id, tenant_id, scope (AGENT/PROJECT/ORG), subject_type, subject_id, fact, source_reference, confidence, created_at, expires_at (nullable).
  3. **Semantic memory** (`memory_embeddings`) — pgvector-backed: embedding_id, tenant_id, memory_fact_id (nullable, may embed raw content instead), content, embedding VECTOR(1536), embedding_model, created_at, with an HNSW index (`vector_cosine_ops`) for approximate nearest-neighbor retrieval at production scale, and a mandatory `embedding_model` column so mixed-model embeddings are never silently compared (a documented and common production failure mode).
- Retrieval flow (clause 22) routes by access pattern: hot per-turn context reads never touch pgvector; only semantic recall queries do — this avoids adding embedding latency to lookups a key-value or relational read can answer directly.

48. Artifact Registry — Full Data Model Closure

- `artifact_registry`: artifact_id, tenant_id, project_id, task_id, agent_run_id, model_run_id, artifact_type (CODE/DOC/DESIGN/CONFIG/REPORT/DATASET), storage_uri, content_hash (immutability proof), version, status (DRAFT/IN_REVIEW/APPROVED/RELEASED/DEPRECATED), reviewed_by, approved_by, git_commit_ref, deployment_ref, created_at.
- Immutability rule: once `status = APPROVED`, the row's `storage_uri` and `content_hash` become append-only — any further change creates a new `artifact_id` with `parent_artifact_id` set, preserving the full lineage chain required by clause 23.

49. Error Handling — Concrete Retry & Backoff Numbers

- Default retry policy for retryable error classes (TOOL_ERROR, MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT, DATABASE_ERROR): bounded exponential backoff, base delay 500 ms, multiplier 2x, jitter ±20%, maximum 5 attempts, maximum total wait 60 seconds before moving to dead-letter / escalation.
- Non-retryable by default: VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, ARTIFACT_ERROR (immutability violations), DEPLOYMENT_ERROR without an approved rollback path — these escalate immediately rather than retrying.
- Idempotency keys are mandatory on every state-mutating task and message so bounded retries never double-execute a side effect (e.g., a duplicate deployment or duplicate external API call).

50. Disaster Recovery — Concrete RPO/RTO Targets

- Database: continuous WAL archiving, point-in-time recovery; **RPO ≤ 5 minutes, RTO ≤ 60 minutes** for the control-plane database.
- Artifacts (object storage): versioned storage with cross-region replication; **RPO ≤ 15 minutes, RTO ≤ 4 hours**.
- Full restore drill required at minimum **quarterly**, with results recorded as a `decision_records` entry (clause 24) including drill date, data restored, time taken, and pass/fail.

51. Configuration & Feature Flags — Full Data Model Closure

- `feature_flags`: flag_id, tenant_id (nullable = global default), flag_key, flag_type (BOOLEAN/PERCENTAGE/VARIANT), default_value, tenant_override_value, environment, status, created_by, updated_at.

- `configuration_versions`: config_id, tenant_id, environment, version, payload, validated (boolean), rollback_of (nullable self-reference), created_by, created_at — every configuration change is a new immutable version, never an in-place update, so rollback is always "activate a previous version," never a destructive edit.

52. API Versioning & Security (Control-Plane Contract Hardening)

- All control-plane endpoints are versioned under `/v1`. Breaking changes require `/v2`; `/v1` is supported for a minimum 12-month deprecation window after `/v2` ships.
- Every endpoint requires bearer JWT authentication (OAuth2 client-credentials for service/agent callers, OAuth2 authorization-code for human users), scoped to a single `tenant_id` claim validated on every request — see the expanded `control_plane_openapi_v1.3.yaml` for the full `securitySchemes` and per-endpoint `requestBody`/response schemas, which were entirely absent from v1.2.
- Standard error envelope on every non-2xx response: `{ "error_code", "message", "correlation_id", "retryable" }`, matching the error classes in clause 31/49.

53. Acceptance Test Matrix — Measurable Version

Each v1.2 checkbox now carries a numeric or binary pass condition instead of an unqualified checkbox: - Agent creation → sandbox → evaluation → approval → activation completes end-to-end in a test tenant in **< 10 minutes** wall-clock for a standard-complexity agent spec. - Model router fallback triggers and completes within **< 3 seconds** of a primary provider health check failing. - A cross-tenant data-access attempt (row belonging to tenant A requested by a tenant B session) is rejected **100% of the time** in an automated test suite of at least 50 adversarial cases. - Workflow state survives a forced process restart mid-workflow and resumes from `workflow_history` with **zero task duplication**, verified by idempotency-key audit. - Disaster recovery drill meets the RPO/RTO numbers in clause 50 on at least the two most recent quarterly drills. - Full audit trail (`audit_events`) reconstructable for any artifact from creation to production release with **no gaps** in `correlation_id` chain, sampled monthly.

54. Interoperability Standards

- Internal tool discovery/invocation: Model Context Protocol (MCP) — consistent with clause 2/Layer 7's existing MCP reference, now made the explicit standard rather than one option among several.
- External agent-to-agent interoperability (only when a specific partner integration is approved): A2A-style HTTP + JSON-RPC capability-card negotiation, disabled by default (clause 42).
- Rationale: standardizing on MCP for tools and an A2A-style contract for external agents avoids building a proprietary protocol that becomes a long-term maintenance and integration liability as the ecosystem consolidates around these patterns.

55. Observability — Agent-Specific Trace Requirements

- Beyond clause 29's logs/metrics/traces/events/alerts: every model routing decision, every policy ALLOW/DENY/REQUIRE_APPROVAL decision, and every tool invocation must emit a structured trace event including the *alternatives considered and rejected* where applicable (e.g., which fallback models were skipped and why) — plain latency/error metrics are insufficient to debug non-deterministic agent behavior in production, a documented gap in standard APM tooling for agentic systems.

56. Indexing & Performance Baseline

- Every foreign key column is indexed by default. Every `tenant_id` column participates in a composite index with the table's primary lookup key (e.g., `(tenant_id, status)` on `task_registry`) since virtually every production query is tenant-scoped.
- `memory_embeddings.embedding` uses an HNSW index (`m = 16`, `ef_construction = 64` as a documented, tunable starting point) rather than IVFFlat, prioritizing recall/latency balance for an interactive agent workload over faster index build time.

57. Compliance & Audit Retention

- `audit_events` retention: minimum 400 days online, then cold-archive for a minimum of 5 years, configurable per tenant policy for regulated environments.
- Every RED-tier action (clause 14) is retained without any automatic expiry.

58. Explicit Non-Goals (Honesty Clause)

- This blueprint still does not include: actual infrastructure-as-code, actual cloud account/network topology, actual provider contracts/pricing agreements, actual penetration-test results, or actual production credentials. These remain environment-specific choices the package's own README already flagged as out of scope in v1.2 — v1.3 does not manufacture fictitious values for these to appear more "complete"; it only closes the *architectural and data-model* gaps that were within a specification document's proper scope.

Section C — Database Schema v1.3 (Full SQL — 27 Tables)

```
-- ======================================================================
-- AI DIGITAL OFFICE OS v1.3 — Full Logical Schema
-- Closes all gaps identified in the v1.2 review: multi-tenancy, RBAC,
-- workflow durability, agent messaging, tiered memory, artifact lineage,
-- secrets references, feature flags. Every business table carries
-- tenant_id; every junction table carries tenant_id in its composite
-- foreign keys so referential integrity alone cannot create a
-- cross-tenant privilege escalation.
-- Target: PostgreSQL 15+ with the pgvector extension.
-- ======================================================================

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS vector;

-- ======================================================================
-- 1. IDENTITY & MULTI-TENANCY  (Blueprint clause 41)
-- ======================================================================

CREATE TABLE organizations (
  tenant_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_name VARCHAR(160) NOT NULL,
  org_slug VARCHAR(80) NOT NULL UNIQUE,
  status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',       -- ACTIVE | SUSPENDED | ARCHIVED
  plan_tier VARCHAR(32) DEFAULT 'STANDARD',
  data_residency VARCHAR(32),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(256) NOT NULL UNIQUE,
  display_name VARCHAR(160),
  auth_provider VARCHAR(64) NOT NULL DEFAULT 'PASSWORD', -- PASSWORD | SSO_OIDC | SSO_SAML
  status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',
  mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- A user may belong to more than one tenant; each membership row is the
-- tenant-scoping anchor for every role assignment below.
CREATE TABLE user_organization_membership (
  membership_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  user_id UUID NOT NULL REFERENCES users(user_id),
  status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',       -- ACTIVE | INVITED | SUSPENDED | REMOVED
  invited_by UUID REFERENCES users(user_id),
  joined_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, user_id)
);

-- Permissions are system-wide: they describe what the software CAN do,
-- independent of any tenant. Tenants compose them into roles.
CREATE TABLE permissions (
  permission_id VARCHAR(96) PRIMARY KEY,              -- e.g. 'agent:create', 'deployment:approve'
  resource VARCHAR(64) NOT NULL,
  action VARCHAR(32) NOT NULL,                        -- READ|WRITE|CREATE|UPDATE|DELETE|EXECUTE|DEPLOY|APPROVE|EXPORT|ADMINISTER
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Roles are tenant-scoped (NULL tenant_id = system template a tenant clones).
CREATE TABLE roles (
  role_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES organizations(tenant_id),  -- NULL = system template
  role_name VARCHAR(96) NOT NULL,
  is_default BOOLEAN NOT NULL DEFAULT FALSE,           -- default role granted on membership join
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, role_name),
  UNIQUE (tenant_id, role_id)
);

-- Composite FK on (tenant_id, role_id) prevents a role from tenant B
-- being attached under tenant A's scope.
CREATE TABLE role_permissions (
  tenant_id UUID,                                      -- mirrors roles.tenant_id (NULL for system roles)
  role_id UUID NOT NULL,
  permission_id VARCHAR(96) NOT NULL REFERENCES permissions(permission_id),
  granted_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (role_id, permission_id),
  FOREIGN KEY (tenant_id, role_id) REFERENCES roles(tenant_id, role_id)
);

CREATE TABLE user_roles (
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  user_id UUID NOT NULL REFERENCES users(user_id),
```

```
  role_id UUID NOT NULL,
  assigned_by UUID REFERENCES users(user_id),
  assigned_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (tenant_id, user_id, role_id),
  FOREIGN KEY (tenant_id, role_id) REFERENCES roles(tenant_id, role_id)
);

-- ============================================================================
-- 2. PROVIDER / MODEL / AGENT REGISTRIES  (Blueprint clauses 6-11, 41)
-- ============================================================================

CREATE TABLE provider_registry (
  provider_id VARCHAR(64) PRIMARY KEY,
  provider_name VARCHAR(128) NOT NULL,
  provider_type VARCHAR(64) NOT NULL,
  adapter_type VARCHAR(128) NOT NULL,
  protocol VARCHAR(64),
  base_endpoint TEXT,
  authentication_method VARCHAR(64),
  supported_capabilities JSONB NOT NULL DEFAULT '[]',
  supported_models JSONB NOT NULL DEFAULT '[]',
  rate_limits JSONB,
  quota_rules JSONB,
  pricing_rules JSONB,
  privacy_classification VARCHAR(32),
  data_residency JSONB,
  availability VARCHAR(32) DEFAULT 'ACTIVE',
  health_status VARCHAR(32) DEFAULT 'UNKNOWN',
  version VARCHAR(32),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE model_registry (
  model_id VARCHAR(64) PRIMARY KEY,
  provider_id VARCHAR(64) NOT NULL REFERENCES provider_registry(provider_id),
  model_name VARCHAR(160) NOT NULL,
  model_version VARCHAR(64),
  capabilities JSONB NOT NULL DEFAULT '[]',
  context_window INT,
  input_types JSONB,
  output_types JSONB,
  tool_calling BOOLEAN DEFAULT FALSE,
  structured_output BOOLEAN DEFAULT FALSE,
  vision BOOLEAN DEFAULT FALSE,
  coding BOOLEAN DEFAULT FALSE,
  reasoning BOOLEAN DEFAULT FALSE,
  research BOOLEAN DEFAULT FALSE,
  latency_profile JSONB,
  cost_profile JSONB,
  privacy_classification VARCHAR(32),
  local_cloud VARCHAR(16),
  availability VARCHAR(32) DEFAULT 'ACTIVE',
  health_status VARCHAR(32) DEFAULT 'UNKNOWN',
  evaluation_score NUMERIC(6,3),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- NULL tenant_id = system agent template a tenant may clone; every
-- executable/running agent row MUST have a non-null tenant_id.
CREATE TABLE agent_registry (
  agent_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID REFERENCES organizations(tenant_id),
  agent_name VARCHAR(128) NOT NULL,
  department VARCHAR(64) NOT NULL,
  role VARCHAR(128) NOT NULL,
  purpose TEXT,
  capabilities JSONB NOT NULL DEFAULT '[]',
  allowed_tools JSONB NOT NULL DEFAULT '[]',
  permissions JSONB NOT NULL DEFAULT '[]',
  data_access JSONB NOT NULL DEFAULT '[]',
  preferred_capabilities JSONB NOT NULL DEFAULT '[]',
  preferred_provider VARCHAR(64) REFERENCES provider_registry(provider_id),
  preferred_model VARCHAR(64) REFERENCES model_registry(model_id),
  fallback_models JSONB NOT NULL DEFAULT '[]',
  input_schema JSONB,
  output_schema JSONB,
  security_level VARCHAR(16) DEFAULT 'GREEN',
  lifecycle_state VARCHAR(32) DEFAULT 'DRAFT',           --
DRAFT|SANDBOX|TESTED|EVALUATED|APPROVED|ACTIVE|UPDATED|DEPRECATED|RETIRED
  status VARCHAR(32) DEFAULT 'INACTIVE',
  version VARCHAR(32) DEFAULT '1.0.0',
  parent_agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  evaluation_score NUMERIC(6,3),
  success_rate NUMERIC(6,3),
  average_latency_ms INT,
  average_cost NUMERIC(12,6),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE prompt_registry (
```

```
  prompt_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID REFERENCES organizations(tenant_id),
  agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  version VARCHAR(32) NOT NULL,
  system_instruction TEXT NOT NULL,
  variables JSONB,
  input_contract JSONB,
  output_contract JSONB,
  evaluation_score NUMERIC(6,3),
  security_classification VARCHAR(32),
  changelog TEXT,
  rollback_version VARCHAR(32),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- ============================================================================
-- 3. POLICY, APPROVAL & ACTIVATION AUTHORITY  (clauses 12-14, 45)
-- ============================================================================

CREATE TABLE policy_registry (
  policy_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID REFERENCES organizations(tenant_id),
  policy_name VARCHAR(160) NOT NULL,
  policy_version VARCHAR(32) NOT NULL,
  rules JSONB NOT NULL,
  status VARCHAR(32) DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE approval_requests (
  request_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64),
  requester VARCHAR(128),
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  action VARCHAR(128) NOT NULL,                          -- includes 'AGENT_ACTIVATE' per clause 45
  risk_level VARCHAR(16) NOT NULL,                       -- GREEN|YELLOW|RED
  reason TEXT,
  approver VARCHAR(128),
  decision VARCHAR(32),
  expires_at TIMESTAMPTZ,
  execution_result JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  decided_at TIMESTAMPTZ
);

-- ============================================================================
-- 4. TASK ENGINE & DURABLE WORKFLOWS  (clauses 17-18, 43)
-- ============================================================================

CREATE TABLE workflow_registry (
  workflow_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64),
  workflow_type VARCHAR(96) NOT NULL,
  definition_version VARCHAR(32) NOT NULL,
  current_state JSONB,
  status VARCHAR(32) DEFAULT 'RUNNING',                  -- RUNNING|PAUSED|COMPLETED|FAILED|CANCELLED|ESCALATED
  started_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  completed_at TIMESTAMPTZ
);

-- Append-only, replayable event log: on process restart, replay this to
-- reconstruct workflow_registry.current_state rather than trusting only
-- the mutable row (durable-execution pattern).
CREATE TABLE workflow_history (
  event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  workflow_id VARCHAR(64) NOT NULL REFERENCES workflow_registry(workflow_id),
  sequence_no BIGINT NOT NULL,
  event_type VARCHAR(64) NOT NULL,                       --
STARTED|TASK_DISPATCHED|TASK_COMPLETED|RETRY|ESCALATED|ROLLED_BACK|COMPLETED
  payload JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (workflow_id, sequence_no)
);

CREATE TABLE task_registry (
  task_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64),
  workflow_id VARCHAR(64) REFERENCES workflow_registry(workflow_id),
  parent_task_id VARCHAR(64) REFERENCES task_registry(task_id),
  assigned_agent VARCHAR(64) REFERENCES agent_registry(agent_id),
  required_capability VARCHAR(128),
  priority VARCHAR(16),
  risk_level VARCHAR(16),
  security_level VARCHAR(16),
  dependencies JSONB DEFAULT '[]',
  input JSONB,
```

```
  expected_output JSONB,
  status VARCHAR(32) DEFAULT 'CREATED',                  --
CREATED|QUEUED|ASSIGNED|RUNNING|WAITING|BLOCKED|FAILED|RETRYING|COMPLETED|CANCELLED|ESCALATED
  retry_count INT DEFAULT 0,
  idempotency_key VARCHAR(128) NOT NULL,
  deadline TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, idempotency_key)
);

-- ======================================================================
-- 5. AGENT-TO-AGENT MESSAGE BUS  (clause 42)
-- ======================================================================

CREATE TABLE agent_messages (
  message_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  workflow_id VARCHAR(64) REFERENCES workflow_registry(workflow_id),
  sender_agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  receiver_agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  message_type VARCHAR(64) NOT NULL,
  purpose TEXT,
  priority VARCHAR(16),
  security_level VARCHAR(16),
  input_payload JSONB,
  expected_output_schema JSONB,
  dependencies JSONB DEFAULT '[]',
  deadline TIMESTAMPTZ,
  artifact_reference VARCHAR(64),
  status VARCHAR(32) DEFAULT 'SENT',                     -- SENT|DELIVERED|ACKNOWLEDGED|FAILED
  result JSONB,
  error JSONB,
  created_at TIMESTAMPTZ DEFAULT now(),
  delivered_at TIMESTAMPTZ,
  acknowledged_at TIMESTAMPTZ
);

-- Optional external interoperability surface (disabled by default; see
-- clause 42/54). Publishes this Office's agents as A2A-discoverable cards.
CREATE TABLE a2a_capability_cards (
  card_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  card_payload JSONB NOT NULL,
  enabled BOOLEAN NOT NULL DEFAULT FALSE,
  published_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- ======================================================================
-- 6. KNOWLEDGE & MEMORY — TIERED  (clause 47)
-- ======================================================================

-- Tier 1: working memory (fast, short-lived; TTL enforced by application
-- job or Postgres partition rotation, default TTL 4 hours).
CREATE TABLE working_memory_cache (
  cache_key VARCHAR(256) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64),
  payload JSONB NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Tier 2: durable structured memory.
CREATE TABLE memory_facts (
  memory_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  scope VARCHAR(16) NOT NULL,                            -- AGENT|PROJECT|ORG
  subject_type VARCHAR(32) NOT NULL,
  subject_id VARCHAR(64) NOT NULL,
  fact TEXT NOT NULL,
  source_reference VARCHAR(128),
  confidence NUMERIC(4,3),
  created_at TIMESTAMPTZ DEFAULT now(),
  expires_at TIMESTAMPTZ
);

-- Tier 3: semantic memory (pgvector, HNSW-indexed).
CREATE TABLE memory_embeddings (
  embedding_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  memory_fact_id UUID REFERENCES memory_facts(memory_id),
  content TEXT NOT NULL,
  embedding VECTOR(1536) NOT NULL,
  embedding_model VARCHAR(96) NOT NULL,                  -- prevents mixing incompatible embedding spaces
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_memory_embeddings_hnsw
  ON memory_embeddings USING hnsw (embedding vector_cosine_ops)
```

```
  WITH (m = 16, ef_construction = 64);

-- ============================================================
-- 7. ARTIFACT REGISTRY  (clause 48)
-- ============================================================

CREATE TABLE artifact_registry (
  artifact_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  agent_run_id VARCHAR(64),
  model_run_id VARCHAR(64),
  artifact_type VARCHAR(32) NOT NULL,                    -- CODE|DOC|DESIGN|CONFIG|REPORT|DATASET
  storage_uri TEXT NOT NULL,
  content_hash VARCHAR(128) NOT NULL,
  version VARCHAR(32) NOT NULL DEFAULT '1.0.0',
  status VARCHAR(32) DEFAULT 'DRAFT',                    -- DRAFT|IN_REVIEW|APPROVED|RELEASED|DEPRECATED
  parent_artifact_id UUID REFERENCES artifact_registry(artifact_id),
  reviewed_by VARCHAR(128),
  approved_by VARCHAR(128),
  git_commit_ref VARCHAR(128),
  deployment_ref VARCHAR(128),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- ============================================================
-- 8. DECISION RECORDS & AUDIT  (clauses 24, 29, 50, 57)
-- ============================================================

CREATE TABLE decision_records (
  decision_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64),
  decision TEXT NOT NULL,
  reason TEXT,
  alternatives JSONB,
  evidence JSONB,
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  model_id VARCHAR(64) REFERENCES model_registry(model_id),
  author VARCHAR(128),
  approval JSONB,
  impact TEXT,
  status VARCHAR(32),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit_events (
  event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES organizations(tenant_id),
  correlation_id VARCHAR(128),
  event_type VARCHAR(128) NOT NULL,
  actor_type VARCHAR(32),
  actor_id VARCHAR(128),
  project_id VARCHAR(64),
  task_id VARCHAR(64),
  workflow_id VARCHAR(64),
  payload JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- ============================================================
-- 9. SECRETS REFERENCES (no raw secrets ever stored — clause 46)
-- ============================================================

CREATE TABLE secrets_vault_references (
  reference_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  secret_name VARCHAR(160) NOT NULL,
  vault_path TEXT NOT NULL,                              -- pointer only, e.g. into an external KMS/Vault
  scope_agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  scope_provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
  scope_tool VARCHAR(96),
  rotation_policy VARCHAR(64),
  last_rotated_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ,
  created_by VARCHAR(128),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- ============================================================
-- 10. CONFIGURATION & FEATURE FLAGS  (clause 51)
-- ============================================================

CREATE TABLE feature_flags (
  flag_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES organizations(tenant_id),    -- NULL = global default
  flag_key VARCHAR(128) NOT NULL,
  flag_type VARCHAR(16) NOT NULL DEFAULT 'BOOLEAN',      -- BOOLEAN|PERCENTAGE|VARIANT
  default_value JSONB NOT NULL,
  tenant_override_value JSONB,
  environment VARCHAR(32) NOT NULL DEFAULT 'production',
  status VARCHAR(32) DEFAULT 'ACTIVE',
```

```
  created_by VARCHAR(128),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, flag_key, environment)
);

CREATE TABLE configuration_versions (
  config_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES organizations(tenant_id),
  environment VARCHAR(32) NOT NULL,
  version VARCHAR(32) NOT NULL,
  payload JSONB NOT NULL,
  validated BOOLEAN NOT NULL DEFAULT FALSE,
  rollback_of UUID REFERENCES configuration_versions(config_id),
  created_by VARCHAR(128),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- ============================================================================
-- 11. INDEXES  (clause 56 — every FK indexed, every tenant_id composite)
-- ============================================================================

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_membership_tenant_user ON user_organization_membership(tenant_id, user_id);
CREATE INDEX idx_roles_tenant ON roles(tenant_id);
CREATE INDEX idx_role_permissions_tenant_role ON role_permissions(tenant_id, role_id);
CREATE INDEX idx_user_roles_tenant_user ON user_roles(tenant_id, user_id);

CREATE INDEX idx_agent_registry_tenant_status ON agent_registry(tenant_id, status);
CREATE INDEX idx_agent_registry_tenant_lifecycle ON agent_registry(tenant_id, lifecycle_state);
CREATE INDEX idx_prompt_registry_tenant_agent ON prompt_registry(tenant_id, agent_id);

CREATE INDEX idx_policy_registry_tenant ON policy_registry(tenant_id, status);
CREATE INDEX idx_approval_requests_tenant_status ON approval_requests(tenant_id, decision);

CREATE INDEX idx_workflow_registry_tenant_status ON workflow_registry(tenant_id, status);
CREATE INDEX idx_workflow_history_tenant_workflow ON workflow_history(tenant_id, workflow_id, sequence_no);
CREATE INDEX idx_task_registry_tenant_status ON task_registry(tenant_id, status);
CREATE INDEX idx_task_registry_tenant_workflow ON task_registry(tenant_id, workflow_id);

CREATE INDEX idx_agent_messages_tenant_receiver ON agent_messages(tenant_id, receiver_agent_id, status);
CREATE INDEX idx_agent_messages_tenant_workflow ON agent_messages(tenant_id, workflow_id);

CREATE INDEX idx_working_memory_tenant_expiry ON working_memory_cache(tenant_id, expires_at);
CREATE INDEX idx_memory_facts_tenant_subject ON memory_facts(tenant_id, subject_type, subject_id);

CREATE INDEX idx_artifact_registry_tenant_project ON artifact_registry(tenant_id, project_id, status);
CREATE INDEX idx_artifact_registry_tenant_task ON artifact_registry(tenant_id, task_id);

CREATE INDEX idx_decision_records_tenant_project ON decision_records(tenant_id, project_id);
CREATE INDEX idx_audit_events_tenant_correlation ON audit_events(tenant_id, correlation_id);
CREATE INDEX idx_audit_events_tenant_created ON audit_events(tenant_id, created_at);

CREATE INDEX idx_secrets_refs_tenant ON secrets_vault_references(tenant_id, scope_agent_id);
CREATE INDEX idx_feature_flags_tenant_key ON feature_flags(tenant_id, flag_key, environment);
CREATE INDEX idx_config_versions_tenant_env ON configuration_versions(tenant_id, environment, created_at);

-- ============================================================================
-- 12. ROW-LEVEL SECURITY BASELINE (defense-in-depth, clause 41/44)
-- Application must SET app.current_tenant_id per session/request; RLS
-- policies below are a second, database-enforced isolation layer that
-- holds even if a service-layer tenant check is ever missed.
-- ============================================================================

ALTER TABLE task_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_task_registry ON task_registry
  USING (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_registry ON agent_registry
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE artifact_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_artifact_registry ON artifact_registry
  USING (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_audit_events ON audit_events
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true));

-- (Extend the same RLS pattern to remaining tenant-scoped tables in the
-- implementation phase; the four above establish the required pattern.)
```

Section D — Control Plane OpenAPI v1.3 (Full YAML — 18 Endpoints)

```
openapi: 3.0.3
info:
  title: AI Digital Office OS Control Plane
  version: 1.3.0
  description: >
```

```yaml
    Governed control-plane API for AI DIGITAL OFFICE OS. Every endpoint is
    tenant-scoped via the JWT 'tenant_id' claim, versioned under /v1, and
    returns the standard error envelope on any non-2xx response (see
    ErrorResponse schema). This closes the v1.2 gap of a skeleton contract
    with no security scheme and no request/response bodies.
servers:
  - url: https://api.digitaloffice.example/v1
    description: Production (placeholder host — real endpoint is an
      environment-specific choice, intentionally not fabricated here)

security:
  - bearerAuth: []

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: >
        OAuth2 client-credentials token for service/agent callers, or
        OAuth2 authorization-code token for human users. Token MUST carry
        a 'tenant_id' claim; every request is scoped to that tenant.

  schemas:
    ErrorResponse:
      type: object
      required: [error_code, message, correlation_id, retryable]
      properties:
        error_code:
          type: string
          enum: [VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, TOOL_ERROR,
                 MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT,
                 DATABASE_ERROR, ARTIFACT_ERROR, DEPLOYMENT_ERROR, UNKNOWN_ERROR]
        message: { type: string }
        correlation_id: { type: string }
        retryable: { type: boolean }

    Task:
      type: object
      required: [task_id, tenant_id, status]
      properties:
        task_id: { type: string }
        tenant_id: { type: string, format: uuid }
        project_id: { type: string }
        workflow_id: { type: string }
        parent_task_id: { type: string }
        assigned_agent: { type: string }
        required_capability: { type: string }
        priority: { type: string }
        risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
        security_level: { type: string }
        dependencies: { type: array, items: { type: string } }
        input: { type: object }
        expected_output: { type: object }
        status:
          type: string
          enum: [CREATED, QUEUED, ASSIGNED, RUNNING, WAITING, BLOCKED, FAILED, RETRYING, COMPLETED, CANCELLED, ESCALATED]
        retry_count: { type: integer }
        idempotency_key: { type: string }
        deadline: { type: string, format: date-time }

    TaskCreateRequest:
      type: object
      required: [project_id, required_capability, input, idempotency_key]
      properties:
        project_id: { type: string }
        workflow_id: { type: string }
        parent_task_id: { type: string }
        required_capability: { type: string }
        priority: { type: string, enum: [LOW, NORMAL, HIGH, CRITICAL] }
        risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
        security_level: { type: string }
        dependencies: { type: array, items: { type: string } }
        input: { type: object }
        expected_output: { type: object }
        idempotency_key:
          type: string
          description: Required — guarantees a retried request never double-executes.
        deadline: { type: string, format: date-time }

    AgentSpecification:
      type: object
      required: [agent_name, department, role, purpose, capabilities]
      properties:
        agent_name: { type: string }
        department: { type: string }
        role: { type: string }
        purpose: { type: string }
        capabilities: { type: array, items: { type: string } }
        required_model_capabilities: { type: array, items: { type: string } }
        allowed_tools: { type: array, items: { type: string } }
        permissions: { type: array, items: { type: string } }
```

```yaml
      data_access: { type: array, items: { type: string } }
      input_schema: { type: object }
      output_schema: { type: object }
      security_level: { type: string, enum: [GREEN, YELLOW, RED], default: GREEN }
      sandbox_policy: { type: object }
      parent_agent_id: { type: string }

    AgentRecord:
      allOf:
        - $ref: '#/components/schemas/AgentSpecification'
        - type: object
          properties:
            agent_id: { type: string }
            tenant_id: { type: string, format: uuid, nullable: true }
            lifecycle_state:
              type: string
              enum: [DRAFT, SANDBOX, TESTED, EVALUATED, APPROVED, ACTIVE, UPDATED, DEPRECATED, RETIRED]
            evaluation_score: { type: number }
            success_rate: { type: number }
            created_at: { type: string, format: date-time }
            updated_at: { type: string, format: date-time }

    RoutingRequest:
      type: object
      required: [task_id, agent_id, required_capability]
      properties:
        task_id: { type: string }
        agent_id: { type: string }
        required_capability: { type: string }
        security_classification: { type: string }
        data_classification: { type: string }
        max_cost: { type: number }
        max_latency_ms: { type: integer }

    RoutingDecision:
      type: object
      properties:
        task_id: { type: string }
        agent_id: { type: string }
        selected_provider: { type: string }
        selected_model: { type: string }
        reason: { type: string }
        policy_result: { type: string, enum: [ALLOW, DENY, REQUIRE_APPROVAL, REQUIRE_ESCALATION] }
        fallback_chain_used: { type: array, items: { type: string } }
        decided_at: { type: string, format: date-time }

    ApprovalRequest:
      type: object
      required: [action, risk_level]
      properties:
        task_id: { type: string }
        agent_id: { type: string }
        action:
          type: string
          description: "Includes AGENT_ACTIVATE for the Factory→Production transition (blueprint clause 45)."
        risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
        reason: { type: string }

    ApprovalRecord:
      allOf:
        - $ref: '#/components/schemas/ApprovalRequest'
        - type: object
          properties:
            request_id: { type: string }
            tenant_id: { type: string, format: uuid }
            requester: { type: string }
            approver: { type: string }
            decision: { type: string, enum: [APPROVED, DENIED, EXPIRED] }
            expires_at: { type: string, format: date-time }
            execution_result: { type: object }
            created_at: { type: string, format: date-time }
            decided_at: { type: string, format: date-time }

    Workflow:
      type: object
      properties:
        workflow_id: { type: string }
        tenant_id: { type: string, format: uuid }
        project_id: { type: string }
        workflow_type: { type: string }
        definition_version: { type: string }
        current_state: { type: object }
        status: { type: string, enum: [RUNNING, PAUSED, COMPLETED, FAILED, CANCELLED, ESCALATED] }
        started_at: { type: string, format: date-time }
        completed_at: { type: string, format: date-time }

    Artifact:
      type: object
      properties:
        artifact_id: { type: string, format: uuid }
        tenant_id: { type: string, format: uuid }
        project_id: { type: string }
        task_id: { type: string }
```

```yaml
        artifact_type: { type: string, enum: [CODE, DOC, DESIGN, CONFIG, REPORT, DATASET] }
        storage_uri: { type: string }
        content_hash: { type: string }
        version: { type: string }
        status: { type: string, enum: [DRAFT, IN_REVIEW, APPROVED, RELEASED, DEPRECATED] }
        parent_artifact_id: { type: string, format: uuid, nullable: true }

    MemoryQueryRequest:
      type: object
      required: [query_text]
      properties:
        query_text: { type: string }
        scope: { type: string, enum: [AGENT, PROJECT, ORG] }
        subject_id: { type: string }
        top_k: { type: integer, default: 10 }
        distance_threshold: { type: number, default: 0.4 }

    MemoryQueryResult:
      type: object
      properties:
        embedding_id: { type: string, format: uuid }
        content: { type: string }
        similarity: { type: number }
        memory_fact_id: { type: string, format: uuid, nullable: true }

    FeatureFlag:
      type: object
      properties:
        flag_id: { type: string, format: uuid }
        flag_key: { type: string }
        flag_type: { type: string, enum: [BOOLEAN, PERCENTAGE, VARIANT] }
        default_value: {}
        tenant_override_value: {}
        environment: { type: string }
        status: { type: string }

    Tenant:
      type: object
      properties:
        tenant_id: { type: string, format: uuid }
        org_name: { type: string }
        org_slug: { type: string }
        status: { type: string, enum: [ACTIVE, SUSPENDED, ARCHIVED] }
        plan_tier: { type: string }

  responses:
    Unauthorized:
      description: Missing/invalid bearer token
      content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }
    PolicyDenied:
      description: Central Policy Engine returned DENY
      content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }
    NotFound:
      description: Resource not found (or not visible to this tenant)
      content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }

paths:
  /tasks:
    post:
      summary: Create a governed task
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TaskCreateRequest' }
      responses:
        '201':
          description: Task created
          content: { application/json: { schema: { $ref: '#/components/schemas/Task' } } }
        '403': { $ref: '#/components/responses/PolicyDenied' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /tasks/{task_id}:
    get:
      summary: Get task state
      parameters:
        - { name: task_id, in: path, required: true, schema: { type: string } }
      responses:
        '200':
          description: Task state
          content: { application/json: { schema: { $ref: '#/components/schemas/Task' } } }
        '404': { $ref: '#/components/responses/NotFound' }

  /agents:
    post:
      summary: Submit an agent specification to Agent Factory
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/AgentSpecification' }
      responses:
        '202':
```

```
          description: Agent creation workflow started (lands in DRAFT, not ACTIVE)
          content: { application/json: { schema: { $ref: '#/components/schemas/AgentRecord' } } }
        '403': { $ref: '#/components/responses/PolicyDenied' }
    get:
      summary: List agents visible to the caller's tenant
      parameters:
        - { name: lifecycle_state, in: query, schema: { type: string } }
        - { name: department, in: query, schema: { type: string } }
      responses:
        '200':
          description: Agent list
          content:
            application/json:
              schema: { type: array, items: { $ref: '#/components/schemas/AgentRecord' } }

  /agents/{agent_id}:
    get:
      summary: Get agent registry record
      parameters:
        - { name: agent_id, in: path, required: true, schema: { type: string } }
      responses:
        '200':
          description: Agent record
          content: { application/json: { schema: { $ref: '#/components/schemas/AgentRecord' } } }
        '404': { $ref: '#/components/responses/NotFound' }

  /agents/{agent_id}/messages:
    get:
      summary: List agent-to-agent messages sent to/from this agent
      parameters:
        - { name: agent_id, in: path, required: true, schema: { type: string } }
        - { name: status, in: query, schema: { type: string } }
      responses:
        '200':
          description: Message list
          content: { application/json: { schema: { type: array, items: { type: object } } } }

  /models/route:
    post:
      summary: Request capability-based model routing
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/RoutingRequest' }
      responses:
        '200':
          description: Routing decision
          content: { application/json: { schema: { $ref: '#/components/schemas/RoutingDecision' } } }
        '403': { $ref: '#/components/responses/PolicyDenied' }

  /approvals:
    post:
      summary: Create an approval request (includes AGENT_ACTIVATE)
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ApprovalRequest' }
      responses:
        '201':
          description: Approval request created
          content: { application/json: { schema: { $ref: '#/components/schemas/ApprovalRecord' } } }
    get:
      summary: List pending approvals for the caller's tenant
      parameters:
        - { name: risk_level, in: query, schema: { type: string, enum: [GREEN, YELLOW, RED] } }
      responses:
        '200':
          description: Approval list
          content:
            application/json:
              schema: { type: array, items: { $ref: '#/components/schemas/ApprovalRecord' } }

  /approvals/{request_id}/decision:
    post:
      summary: Record an approval decision (management/Owner action)
      parameters:
        - { name: request_id, in: path, required: true, schema: { type: string } }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [decision]
              properties:
                decision: { type: string, enum: [APPROVED, DENIED] }
                reason: { type: string }
      responses:
        '200':
          description: Updated approval record
          content: { application/json: { schema: { $ref: '#/components/schemas/ApprovalRecord' } } }
```

```yaml
  /providers/health:
    get:
      summary: Provider health status
      responses:
        '200':
          description: Provider health
          content: { application/json: { schema: { type: array, items: { type: object } } } }

  /workflows/{workflow_id}:
    get:
      summary: Get durable workflow state (reconstructed from workflow_history on demand)
      parameters:
        - { name: workflow_id, in: path, required: true, schema: { type: string } }
      responses:
        '200':
          description: Workflow state
          content: { application/json: { schema: { $ref: '#/components/schemas/Workflow' } } }
        '404': { $ref: '#/components/responses/NotFound' }

  /workflows/{workflow_id}/pause:
    post:
      summary: Pause a running workflow
      parameters:
        - { name: workflow_id, in: path, required: true, schema: { type: string } }
      responses:
        '200': { description: Workflow paused }

  /workflows/{workflow_id}/resume:
    post:
      summary: Resume a paused workflow
      parameters:
        - { name: workflow_id, in: path, required: true, schema: { type: string } }
      responses:
        '200': { description: Workflow resumed }

  /artifacts:
    get:
      summary: List artifacts for a project
      parameters:
        - { name: project_id, in: query, required: true, schema: { type: string } }
        - { name: status, in: query, schema: { type: string } }
      responses:
        '200':
          description: Artifact list
          content:
            application/json:
              schema: { type: array, items: { $ref: '#/components/schemas/Artifact' } }

  /artifacts/{artifact_id}:
    get:
      summary: Get artifact record and lineage
      parameters:
        - { name: artifact_id, in: path, required: true, schema: { type: string, format: uuid } }
      responses:
        '200':
          description: Artifact record
          content: { application/json: { schema: { $ref: '#/components/schemas/Artifact' } } }
        '404': { $ref: '#/components/responses/NotFound' }

  /memory/query:
    post:
      summary: Tiered memory retrieval (routes to working/durable/semantic tier)
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/MemoryQueryRequest' }
      responses:
        '200':
          description: Ranked memory results
          content:
            application/json:
              schema: { type: array, items: { $ref: '#/components/schemas/MemoryQueryResult' } }

  /feature-flags:
    get:
      summary: List feature flags visible to the caller's tenant
      responses:
        '200':
          description: Flag list
          content:
            application/json:
              schema: { type: array, items: { $ref: '#/components/schemas/FeatureFlag' } }
    post:
      summary: Create or override a feature flag for this tenant
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/FeatureFlag' }
      responses:
        '201':
```

```
                description: Flag created/updated
                content: { application/json: { schema: { $ref: '#/components/schemas/FeatureFlag' } } }

        /tenants/current:
          get:
            summary: Get the calling token's own tenant record
            responses:
              '200':
                description: Tenant record
                content: { application/json: { schema: { $ref: '#/components/schemas/Tenant' } } }

        /secrets/{reference_id}/rotate:
          post:
            summary: Trigger rotation of a referenced secret (never returns the raw value)
            parameters:
              - { name: reference_id, in: path, required: true, schema: { type: string, format: uuid } }
            responses:
              '202': { description: Rotation triggered — audited as a secret-access event }
              '403': { $ref: '#/components/responses/PolicyDenied' }
```

Section E — Architecture & Sequence Diagrams v1.3

AI DIGITAL OFFICE OS v1.3 Architecture Diagrams

All three v1.2 diagrams are preserved unchanged below, followed by four new sequence/structure diagrams that close the “no sequence diagrams” gap flagged in the v1.2 review.

AI DIGITAL OFFICE OS v1.2/v1.3 Architecture (unchanged from v1.2, still authoritative)

```
flowchart TD
    O[Owner / Chairman] --> CEO[Master CEO]
    CEO --> EM[Executive Manager]
    EM --> AM[Assistant Manager]
    AM --> CP[Orchestration Control Plane]

    CP --> AF[Agent Factory]
    AF --> AR[Agent Registry]
    AF --> SB[Sandbox]
    AF --> EV[Agent Evaluation]
    AR --> AP[Specialized Agent Pool]

    AP --> PE[Central Policy Engine]
    PE --> AUTH[RBAC / ABAC / Permissions]

    AP --> MR[Model Router]
    MR --> MREG[Model Registry]
    MREG --> PG[Provider Gateway]
    PG --> PA[Provider Adapters]
    PA --> P1[OpenAI/GPT]
    PA --> P2[Anthropic/Claude]
    PA --> P3[Google/Gemini]
    PA --> P4[Moonshot/Kimi]
    PA --> P5[Local Models]
    PA --> PF[Future Providers]

    AP --> TG[MCP / Tool Gateway]
    TG --> G[Git]
    TG --> D[Docker]
    TG --> DB[Postgres]
    TG --> FS[Files]
    TG --> WEB[Browsers / Web]
    TG --> AUTO[Automation / n8n]

    CP --> WM[Workflow + Task Engine]
    WM --> DAG[Dependency DAG]

    AP --> MEM[Memory / Knowledge]
    MEM --> V[pgvector]
    AP --> ART[Artifact Registry]
    ART --> GIT[Version Control]

    CP --> QA[Independent QA]
    CP --> SEC[Security]
    CP --> OBS[Observability]
    CP --> COST[Cost Governance]

    AP --> PFAC[Project Factory]
    PFAC --> DEV[Development]
    PFAC --> TEST[Testing]
    PFAC --> REL[Release]
    REL --> MON[Monitoring]

    PE --> APP[Approval Engine]
    APP --> O
```

Agent Factory lifecycle (unchanged from v1.2)

```
flowchart LR
R[Request] --> S[Specification]
S --> B[Sandbox]
B --> T[Tests]
T --> E[Evaluation]
E --> SEC[Security]
SEC --> A[Approval]
A --> REG[Registry]
REG --> ACT[Active]
ACT --> U[Update]
U --> E
ACT --> DEP[Deprecate]
DEP --> RET[Retire]
```

Software Factory lifecycle (unchanged from v1.2)

```
flowchart LR
I[Idea] --> R[Research]
R --> PRD[Requirements / PRD]
PRD --> ARC[Architecture]
ARC --> UX[UI/UX]
UX --> DEV[Build]
DEV --> REV[Code Review]
REV --> QA[QA]
QA --> SEC[Security]
SEC --> STG[Staging]
STG --> APP[Approval]
APP --> PROD[Production]
PROD --> OBS[Monitoring]
OBS --> MAINT[Maintenance]
MAINT --> I
```

NEW IN v1.3 — Sequence & Structure Diagrams

1. Task Routing Sequence (clauses 10, 17, 43, 49)

```
sequenceDiagram
    participant T as Task Engine
    participant PE as Policy Engine
    participant MR as Model Router
    participant MREG as Model Registry
    participant PG as Provider Gateway
    participant P as AI Provider

    T->>PE: Evaluate(agent, task, data_class, risk)
    PE-->>T: ALLOW | DENY | REQUIRE_APPROVAL
    alt ALLOW
        T->>MR: Route(required_capability, policy_result)
        MR->>MREG: Lookup candidate models by capability
        MREG-->>MR: Ranked candidates (health, cost, latency)
        MR->>PG: Invoke selected provider/model
        PG->>P: Request (scoped short-lived credential)
        alt Provider healthy
            P-->>PG: Response
            PG-->>MR: Result
        else Provider failure / timeout
            PG-->>MR: Error (retryable)
            MR->>MR: Bounded backoff (base 500ms, x2, max 5 attempts)
            MR->>PG: Try Approved Secondary
            PG->>P: Request (fallback provider)
            P-->>PG: Response
            PG-->>MR: Result
        end
        MR-->>T: Routing decision + result (fully audited)
    else DENY
        T-->>T: Task marked FAILED, error_code=POLICY_ERROR
    else REQUIRE_APPROVAL
        T->>PE: Create approval_requests row
        PE-->>T: Task status = WAITING
    end
```

2. Agent Creation → Activation Sequence (clauses 4, 5, 14, 45)

```
sequenceDiagram
    participant U as Requester (human or governed agent)
```

```
    participant AF as Agent Factory
    participant SB as Sandbox
    participant EV as Evaluation Engine
    participant SEC as Security Review
    participant APR as Approval Engine
    participant AR as Agent Registry

    U->>AF: Submit AgentSpecification
    AF->>AF: lifecycle_state = DRAFT
    AF->>SB: Deploy to sandbox (no prod credentials)
    SB-->>AF: lifecycle_state = SANDBOX
    AF->>EV: Run evaluation suite
    EV-->>AF: lifecycle_state = TESTED / EVALUATED (score recorded)
    AF->>SEC: Security review
    SEC-->>AF: Pass/Fail
    AF->>APR: Request approval (action=AGENT_ACTIVATE, risk=per policy)
    Note over APR: Factory CANNOT self-activate — clause 45
    APR-->>AF: lifecycle_state = APPROVED (if approved)
    APR->>AR: On explicit activation trigger: lifecycle_state = ACTIVE
    AR-->>U: Agent is now callable in production
```

3. Approval / Risk-Tier Escalation Flow (clause 14)

```
sequenceDiagram
    participant A as Requesting Agent
    participant PE as Policy Engine
    participant APR as Approval Engine
    participant MGR as Designated Manager (YELLOW)
    participant OWN as Owner/Chairman (RED)

    A->>PE: Attempt action
    PE-->>A: Risk classification (GREEN | YELLOW | RED)
    alt GREEN
        PE-->>A: Autonomous execution permitted
    else YELLOW
        PE->>APR: Create approval_requests row
        APR->>MGR: Route for management review
        MGR-->>APR: Decision (APPROVED/DENIED)
        APR-->>A: Execute or block per decision
    else RED
        PE->>APR: Create approval_requests row
        APR->>OWN: Route for Owner sign-off
        OWN-->>APR: Decision (APPROVED/DENIED)
        APR-->>A: Execute or block per decision
    end
    APR->>APR: Write audit_events (full trail, unlimited retention for RED)
```

4. Multi-Tenant Isolation (structural, clause 41/44)

```
flowchart TD
    subgraph Tenant A
        UA[Users A] --> MA[Membership A]
        MA --> RA[Roles A]
        RA --> RPA[role_permissions A]
        UA --> TAA[task_registry rows: tenant_id=A]
        UA --> AGA[agent_registry rows: tenant_id=A]
    end

    subgraph Tenant B
        UB[Users B] --> MB[Membership B]
        MB --> RB[Roles B]
        RB --> RPB[role_permissions B]
        UB --> TAB[task_registry rows: tenant_id=B]
        UB --> AGB[agent_registry rows: tenant_id=B]
    end

    PERM[permissions catalog — system-wide, tenant-agnostic]
    RPA --> PERM
    RPB --> PERM

    RLS[Row-Level Security: app.current_tenant_id]
    TAA -.enforced by.-> RLS
    TAB -.enforced by.-> RLS
    AGA -.enforced by.-> RLS
    AGB -.enforced by.-> RLS

    note1[Composite FKs carry tenant_id through every junction table,\nso a syntactically valid cross-tenant reference is
rejected\nby the database itself, not only by application code.]
```

Section F — Agent Specification Template v1.3

Agent Specification Template v1.3

agent_id: tenant_id: # NULL only for a system-wide template; every executable agent must set this agent_name: department: role: purpose:

Capabilities

Required Model Capabilities

Tools (declared via MCP — Model Context Protocol, blueprint clause 54)

- tool_name: mcp_server: allowed_actions:

Permissions

Data Classification Allowed

- PUBLIC
- INTERNAL

Input Contract

{}

Output Contract

{}

Security Level

GREEN | YELLOW | RED

Sandbox Policy

Describe filesystem, network, database, shell and credential restrictions. Default sandbox credential lifetime: 15 minutes (AI provider keys), 24 hours (tool-gateway service credentials) — see blueprint clause 46.

Idempotency

Every task this agent executes must supply an idempotency_key; the agent must not perform a non-idempotent side effect (deployment, external write, message send) without one.

Evaluation Suite

- Accuracy
- Instruction following
- Tool precision
- Schema compliance
- Security
- Reliability
- Cost
- Latency
- Domain invariants

Lifecycle

DRAFT → SANDBOX → TESTED → EVALUATED → APPROVED → ACTIVE → UPDATED → DEPRECATED → RETIRED

Note (v1.3, blueprint clause 45): reaching APPROVED is the Agent Factory’s maximum authority. The APPROVED → ACTIVE transition requires a separate approval_requests record (action=AGENT_ACTIVATE) — the Factory pipeline cannot activate its own output into production.

External Interoperability (optional, disabled by default)

a2a_capability_card_enabled: false # blueprint clause 42/54 — only for # approved external partner integrations

Owner / Governance

Define who can approve activation and production permissions for this agent, scoped to tenant_id above.

Section G — Non-Functional Requirements & SLOs (New in v1.3)

AI DIGITAL OFFICE OS v1.3 — Non-Functional Requirements & Service Level Objectives

This file is new in v1.3. It exists because the v1.2 review found every non-functional requirement stated only as policy language (“bounded”, “auditable”, “scalable”) with no concrete number to test against. Every number below is a starting baseline meant to be tuned during Phase 1 load testing, not a marketing claim — treat these as the acceptance thresholds referenced in blueprint clause 53.

Availability

- Control plane API: target 99.9% monthly uptime (≈ 43 minutes allowed downtime/month).
- Agent execution layer: target 99.5% monthly uptime (agents may legitimately queue during provider failover; this is looser than the control plane by design).

Latency

- Control-plane read endpoints (`GET /tasks/{id}` , `GET /agents/{id}` , etc.): p95 < 300 ms, p99 < 800 ms.
- Task creation (`POST /tasks`) including Policy Engine evaluation: p95 < 1.2 s.
- Model routing decision (`POST /models/route`) excluding the downstream model call itself: p95 < 400 ms.
- Fallback trigger on primary provider failure: < 3 seconds from health-check failure to fallback provider invocation (blueprint clause 53).
- Memory semantic query (`POST /memory/query` , HNSW-indexed): p95 < 150 ms at up to 10M embedding rows per tenant shard.

Reliability — Retry & Backoff (blueprint clause 49)

- Retryable error classes: base delay 500 ms, multiplier 2x, jitter ±20%, max 5 attempts, max total wait 60 s, then dead-letter/escalate.
- Non-retryable error classes (VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, ARTIFACT_ERROR immutability violations): escalate immediately, zero retries.
- All state-mutating operations require an idempotency key; duplicate delivery of the same key within a 24-hour window is a no-op returning the original result.

Scalability Targets (initial baseline, re-test each phase)

- 200 active agents per tenant (default policy ceiling, clause 5), horizontally scalable per tenant beyond that via explicit policy override.
- 10,000 concurrent tasks in RUNNING/QUEUED state across the platform before horizontal scale-out of the Task Engine is required.
- 1,000 tenants on shared schema before evaluating a shard-per-tenant-group split (the shared-schema/tenant_id pattern scales to tens of thousands of tenants per current practice; 1,000 is a conservative first re-evaluation point, not a hard ceiling).

Disaster Recovery (blueprint clause 50)

- Control-plane database: RPO ≤ 5 minutes, RTO ≤ 60 minutes.
- Artifact object storage: RPO ≤ 15 minutes, RTO ≤ 4 hours.
- Full restore drill: minimum quarterly, logged as a decision_records entry.

Security

- Credential lifetime: AI provider keys issued to a running task, 15 minutes; tool-gateway service credentials, 24 hours (clause 46).
- Cross-tenant isolation: 100% rejection rate on adversarial cross-tenant access test suite (minimum 50 cases), re-run on every schema migration (clause 53).
- Audit event retention: 400 days online, 5 years cold-archive minimum; RED-tier actions retained indefinitely (clause 57).

Cost Governance

- Per-tenant daily and monthly budget enforcement with three tiers: WARNING at 70% of budget, SOFT_LIMIT at 90% (routes to lower-cost models automatically where policy allows), HARD_LIMIT at 100% (blocks further spend pending Owner approval).

Observability

- Every request carries a correlation_id propagated across all services it touches (clause 29).
- Trace retention for agent decision paths (model chosen, alternatives rejected, policy result): minimum 90 days queryable (clause 55).

Self-Healing Bound

- Maximum 3 automatic self-healing attempts per failure before mandatory human escalation (clause 32, unchanged from v1.1/v1.2).

Section H — Implementation Acceptance Checklist v1.3 (Measurable)

AI DIGITAL OFFICE OS v1.3 Implementation Acceptance Checklist

(Measurable version — every item now has a pass condition, not just a checkbox)

Architecture

- [ ] v1.1/v1.2 architecture preserved — all 9 layers (incl. new Layer 9: Identity & Tenancy) present and connected
- [ ] Agent Factory integrated, stops at APPROVED (cannot self-activate — clause 45)
- [ ] Provider Gateway integrated with at least 2 live provider adapters passing health checks
- [ ] Model Registry integrated, router fallback verified to trigger in < 3 seconds (NFR doc)
- [ ] Policy Engine integrated, returns ALLOW/DENY/REQUIRE_APPROVAL/REQUIRE_ESCALATION on 100% of evaluated test cases
- [ ] Approval Engine integrated with all 3 risk tiers (GREEN/YELLOW/RED) demonstrably routed to correct approver

Multi-Tenancy & Identity (new in v1.3)

- [ ] `organizations`, `users`, `user_organization_membership`, `roles`, `permissions`, `role_permissions`, `user_roles` all present
- [ ] Every tenant-scoped table carries `tenant_id`; composite FKs verified on all junction tables
- [ ] Row-Level Security policies active on at minimum: task_registry, agent_registry, artifact_registry, audit_events
- [ ] Adversarial cross-tenant access test suite (≥ 50 cases) passes at 100% rejection rate

Agent Platform

- [ ] Agent creation → sandbox → evaluation → approval → activation completes end-to-end in < 10 minutes (test tenant, standard-complexity spec)
- [ ] Clone/extend/specialize preserves parent_agent_id lineage
- [ ] Sandbox verified to withhold unrestricted production credentials (automated check, not manual review)
- [ ] Evaluation suite produces a numeric evaluation_score for every agent before APPROVED
- [ ] Lifecycle states enforced as a strict state machine (illegal transitions rejected, e.g. DRAFT → ACTIVE directly)
- [ ] Registry queryable by tenant, department, lifecycle_state
- [ ] Versioning: agent updates create a new version, not an in-place overwrite of an ACTIVE agent

AI Platform

- [ ] Provider Registry, adapters, health probes operational
- [ ] Model Registry with capability tags (HEAVY_CODING, DEEP_RESEARCH, etc.)
- [ ] Model evaluation scores versioned and re-runnable
- [ ] Intelligent routing decisions 100% auditable (task_id, agent_id, reason, policy_result all populated)
- [ ] Fallback chain (Primary → Secondary → Local → Human escalation) tested under simulated provider outage
- [ ] Cost controls: WARNING/SOFT_LIMIT/HARD_LIMIT tiers demonstrably trigger at 70%/90%/100% of budget

Security

- [ ] RBAC/ABAC enforced at both gateway (coarse) and service layer (fine-grained)
- [ ] Least privilege: default-deny verified — a role with zero permissions can perform zero actions
- [ ] Secrets vault: zero raw secret values found in application database (automated scan)

- [ ] Data classification (PUBLIC→RESTRICTED) enforced on every external model call
- [ ] Environment isolation (dev/staging/prod) verified — no shared credentials across environments
- [ ] Audit trail reconstructable end-to-end for a sampled artifact with zero correlation_id gaps

Engineering

- [ ] Task engine: all 11 states (CREATED→ESCALATED) reachable and tested
- [ ] Durable workflows: forced process restart mid-workflow resumes from workflow_history with zero task duplication
- [ ] Dependency DAG: circular dependency and deadlock detection verified with synthetic test cases
- [ ] Git controls: protected production branch, no direct push, PR + review required
- [ ] CI/CD: unit, integration, static analysis, security scan all gate merge
- [ ] QA: independent QA pass required before staging promotion
- [ ] Security scanning: zero unresolved CRITICAL findings before production
- [ ] Deployment rollback: tested rollback completes in < RTO target for the affected component

Memory & Artifacts (new in v1.3)

- [ ] Working memory TTL enforced (default 4h), verified expired rows are not returned
- [ ] Durable memory facts queryable by scope (AGENT/PROJECT/ORG)
- [ ] Semantic memory (pgvector/HNSW) query p95 < 150 ms at target scale (NFR doc)
- [ ] embedding_model column enforced — mixed-model embedding comparison blocked
- [ ] Artifact immutability: APPROVED artifact's storage_uri/content_hash cannot be mutated (attempted mutation creates new artifact_id instead)
- [ ] Full lineage reconstructable: Owner Command → ... → Release, for a sampled artifact

Agent Messaging (new in v1.3)

- [ ] agent_messages delivery is at-least-once with verified idempotent handling on message_id
- [ ] A2A external interoperability confirmed disabled by default (a2a_capability_cards.enabled = false unless explicitly turned on per integration)

Operations

- [ ] Logs, metrics, traces, events, alerts cover all 9 layers
- [ ] Agent-specific trace events capture rejected alternatives (model routing, policy decisions) — not just latency/error
- [ ] Backups: automated, tested restore performed at least once before go-live
- [ ] Disaster recovery: RPO ≤ 5 min / RTO ≤ 60 min (database), RPO ≤ 15 min / RTO ≤ 4 hr (artifacts) demonstrated in a drill
- [ ] Feature flags: staged rollout tested (tenant override vs. global default)

Extensibility

- [ ] Add provider without core schema/API rewrite — demonstrated with a second live adapter
- [ ] Add model without core rewrite
- [ ] Add agent without core rewrite
- [ ] Add tool without core rewrite (MCP-declared)
- [ ] Add product type to Project Factory without core rewrite
- [ ] Add tenant without any schema migration (pure data insert into organizations)

Section I — Changelog: v1.2 → v1.3

Changelog — v1.2 → v1.3

Why this version exists

A full A-to-Z review of the v1.2 package found the blueprint's *narrative* made promises (multi-tenancy readiness, artifact lineage, agent messaging, memory, durable workflows, RBAC, secrets management, feature flags) that the v1.2 SQL schema and OpenAPI contract did not actually implement. v1.3 closes that gap. Nothing in v1.2 was removed or weakened; v1.3 only adds.

Blueprint (MD)

- Added: Clauses 41–58 (18 new clauses), each directly answering one gap found in the v1.2 review.

- Unchanged: Clauses 1–40, reproduced verbatim.
- Added: Layer 9 (Identity & Tenancy) to the architecture layer list.

Database schema (SQL)

- v1.2 had 9 tables. v1.3 has 27 tables. New tables:
  - `organizations`, `users`, `user_organization_membership`, `roles`, `permissions`, `role_permissions`, `user_roles` (RBAC + tenancy)
  - `workflow_registry`, `workflow_history` (durable workflow engine)
  - `agent_messages`, `a2a_capability_cards` (agent-to-agent messaging)
  - `working_memory_cache`, `memory_facts`, `memory_embeddings` (tiered memory, pgvector/HNSW)
  - `artifact_registry` (artifact lineage — did not exist in v1.2)
  - `secrets_vault_references` (secrets — did not exist in v1.2; stores pointers only, never raw secrets)
  - `feature_flags`, `configuration_versions` (did not exist in v1.2)
- Changed: every existing v1.2 table (`agent_registry`, `task_registry`, `policy_registry`, `approval_requests`, `prompt_registry`, `decision_records`, `audit_events`) now carries `tenant_id`.
- Added: indexes on every foreign key and every `tenant_id` composite lookup (v1.2 had zero indexes beyond primary keys).
- Added: baseline Row-Level Security policies on 4 core tables as a second, database-enforced tenant-isolation layer.

API contract (OpenAPI)

- v1.2 had 6 endpoints, no security scheme, no request bodies, no error schema. v1.3 has 18 endpoints, JWT bearer security, 15 reusable component schemas, request bodies on every write endpoint, and a standard error envelope.
- New endpoint groups: `/workflows/*`, `/artifacts/*`, `/memory/query`, `/feature-flags`, `/tenants/current`, `/secrets/{id}/rotate`, `/agents/{id}/messages`, `/approvals/{id}/decision`.
- Added: `/v1` path versioning (v1.2 had no version prefix at all).

Diagrams

- v1.2 had 3 flowcharts, all preserved unchanged.
- Added: 4 sequence/structure diagrams (task routing, agent creation→activation, approval escalation, multi-tenant isolation) — v1.2 had zero sequence diagrams.

New standalone files (did not exist in v1.2)

- `nfr_slo_v1.3.md` — concrete latency, availability, retry/backoff, RTO/RPO, and cost-governance numbers, replacing policy-only language.
- `CHANGELOG_v1.2_to_v1.3.md` — this file.

Explicitly NOT done (honesty — see blueprint clause 58)

- No line-by-line diff was run between the two source v1.1/v1.2 PDFs and the v1.2 Markdown; the Markdown was treated as the source of truth per the v1.2 package's own README build-order instructions.
- No infrastructure, cloud topology, provider contracts, pricing, penetration-test results, or production credentials were added or fabricated. These remain out of scope for a specification document.
- No code implementation was produced — this is still a specification/blueprint package, not an application. Handing it to a coding agent (e.g. Claude Code) as a master implementation prompt is the next step, not part of this package.

Section J — Final Honesty Statement

This v1.3 package is now internally consistent: every promise made in the blueprint narrative (multi-tenancy, RBAC, artifact lineage, agent messaging, tiered memory, durable workflows, secrets management, feature flags) has a corresponding table in the schema and, where relevant, an endpoint in the API contract. All numeric NFR/SLO targets are stated as tunable starting baselines for Phase 1 load testing, not measured production results — they have not been benchmarked against a running system because no running system exists yet; this remains a specification and data-model package, not application code. Real infrastructure, provider contracts, pricing, penetration-test results and production credentials are still environment-specific choices for the implementation phase and have not been fabricated here.

Recommendation: this version is suitable to lock as the source for a single English master implementation prompt to hand to a coding agent, using Sections B, C, D, F, G and H as the prompt's source material, in that order.